

Tökéletes. A **RegisterView.vue** a logika "másik fele", amely szorosan együttműködik a Login oldallal (például az átirányítás révén). A struktúra itt is követi a jól bevált mintákat, de az űrlap és a validáció komplexebb a több mező miatt.

Itt a dokumentáció a kód alapján:

RegisterView.vue

1. Áttekintés

A **RegisterView** az új felhasználók regisztrációs felülete. Ez az oldal teszi lehetővé a fiók létrehozását a rendszerben. Mivel a regisztráció több adatot igényel, a komponens kiemelt figyelmet fordít a valós idejű validációra és a felhasználói hibák megelőzésére.

- **Típus:** Page Component (Auth)
- **Útvonal:** `/register` (feltételezett)
- **Szülő Layout:** `AuthLayout.vue`

2. Felépítés és Architektúra

A komponens felépítése megegyezik a többi hitelesítési oldallal, biztosítva a vizuális konzisztenciát. A "Smart Component" megközelítés itt is érvényesül: a `RegisterView` kezeli a logikát, míg a megjelenítést a közös komponensek végzik.

2.1 Komponens Hierarchia

A diagram a `RegisterView` szerkezetét mutatja. Mivel az `AuthLayout` vissza gombja itt sincs aktiválva, a navigációt az űrlap alján lévő gomb biztosítja.

```
graph TD
    %% Fő nézet
    Page[RegisterView.vue]

    %% Layout réteg
    Page -- "Használja" --> Layout[AuthLayout.vue]

    %% Layout belső elemei
    subgraph "AuthLayout Belső Elemei"
        direction TB
        Layout --> Bg[AnimatedBackground]
        Layout --> Theme[ThemeSwitcher]
    end

    %% Slot tartalom
    subgraph "Regisztrációs Űrlap (Slot)"
        direction TB
        Form[Form Element]
    end
```

```

Form --> InpUser[FormInput: Felhasználónév]
Form --> InpEmail[FormInput: Email]
Form --> InpName[FormInput: Teljes név]
Form --> InpPass[FormInput: Jelszó]

Form --> Msg[MessageDisplay]

Form --> BtnReg[BaseButton: Regisztráció]
Form --> BtnLogin[BaseButton: Vissza a belépéshez]
end

%% Kapcsolat
Layout --"Renderelés"--> Form

%% Stílusok
classDef main fill:#42b883,stroke:#35495e,color:white,font-weight:bold;
classDef sub fill:#35495e,stroke:#42b883,color:white;
classDef slot fill:#fff3e0,stroke:#f57c00,stroke-dasharray: 5 5;

class Page,Layout main;
class Bg,Theme,InpUser,InpEmail,InpName,InpPass,Msg,BtnReg,BtnLogin sub;
class Form slot;

```

3. Üzleti Logika (<script setup>)

A logika a `useAuth` composable `register` függvényére épül. A legfontosabb különbség a Login oldalhoz képest, hogy sikeres regisztráció esetén nem a Dashboardra, hanem a Login oldalra irányítjuk a felhasználót.

3.1 Folyamatvezérlés (Flowchart)

Az alábbi ábra bemutatja, hogyan kezeli a rendszer a regisztrációs kérést, beleértve a validációt és a visszajelzéseket.

```

flowchart TD
    Start((Start)) --> Input[/Adatok kitöltése<br>(4 mező)/]
    Input --> Submit[Regisztráció gomb]
    Submit --> Validate{Helyes a formátum?}
    Validate -- NEM --> ShowLocalError[Figyelmeztetés:<br>Helytelen vagy hiányzó adatok]
    ShowLocalError --> Input
    Validate -- IGEN --> Loading[Homokóra indítása...]
    Loading --> APICall[Adatok küldése a szervernek]
    APICall --> ServerCheck{Sikeres létrehozás?}

```

```

ServerCheck -- NEM --> Translate["Hiba fordítása<br>(pl. Foglalt email)"]
Translate --> AuthError["Piros hibaüzenet megjelenítése"]
AuthError --> Input

ServerCheck -- IGEN --> Redirect[/"Átírányítás a Login oldalra<br>(+ siker
paraméter)"/]
Redirect --> End((Vége))

%% Stílusok
style Validate fill:#f9f,stroke:#333
style ServerCheck fill:#bbf,stroke:#333
style Redirect fill:#dfd,stroke:#333,stroke-width:2px

```

3.2 Validáció és Szabályok

Mivel ez a legösszetettebb űrlap, szigorú validációs szabályok (**rules**) érvényesülnek, melyek a **validation-messages.js**-ből származnak:

1. **Felhasználónév (username)**: Egyedinek és megfelelő hosszúságúnak kell lennie.
2. **Email (email)**: Szabványos email formátum.
3. **Teljes név (fullName)**: Kötelező mező (a profil megjelenítéshez).
4. **Jelszó (password)**: Erős jelszó követelmények (hossz, karaktertípusok).

A **validateField** függvény gondoskodik róla, hogy a felhasználó azonnal visszajelzést kapjon (**@blur** eseményre), ha valamit elrontott, még mielőtt elküldené az űrlapot.

3.3 Navigáció és Átírányítás

Sikeres regisztráció esetén a kód nem jelenít meg helyi sikerüzenetet, hanem azonnal átírányít:

```
navigateToLogin({ registered: '1' })
```

Ez magyarázza a **LoginView**-nál látott **checkQueryMessages** logikát: a regisztrációs oldal "átadja a labdát" a bejelentkezési oldalnak, amely az URL paraméter (**registered=1**) alapján írja ki a felhasználónak, hogy *"Sikeres regisztráció, most már beléphetsz"*.

4. Felhasználói Felület (<template>)

A template négy **FormInput** komponenst tartalmaz, mindegyik saját validációs állapottal (**:error**).

- **Mezők:**
- **username** (text)
- **email** (email)

- `fullName` (text)
- `password` (password - rejtett)
- **Visszajelzés:** A `MessageDisplay` komponens itt csak a szerveroldali hibák (pl. "Ez az email cím már foglalt") megjelenítésére szolgál, mivel a siker esetén átirányítás történik.

5. Stílus (CSS)

Ahogy a többi hitelesítési oldalnál, itt is az `@/assets/auth-common.css` biztosítja az egységes megjelenést. A `.register-container` osztály gondoskodik a megfelelő térközökről, hogy a hosszabb űrlap is áttekinthető maradjon mobilon és desktopon egyaránt.